

计算机系统实验报告

组长：张诗琪 组员：高荧，贾培鑫

东北大学计算机科学与工程学院人工智能（一）2302 班

摘要

本文基于“龙芯杯”竞赛背景，从仅支持 4 条指令的基础 CPU 模板出发，设计并实现了一个支持 30 余条 MIPS 指令、具备完整冲突解决机制的五级流水线 CPU。实验采用模块化分工，三人团队分别负责指令译码与数据前递（ID 段）、执行与访存通路（EX/MEM 段）以及全局控制与集成（CTRL 段），最终成功通过了所有功能测试。本工作不仅验证了五级流水线、数据前递、流水线停顿等核心体系结构概念，更通过从理论到工程的完整实践，深化了对 CPU 微架构设计、硬件描述语言（Verilog）及系统级调试的理解，为后续深入探索计算机体系结构奠定了基础。

关键词 五级流水线；MIPS；数据前递；流水线停顿；Verilog；CPU 设计

1. 引言

在本次计算机系统实验中，以“龙芯杯”竞赛为导向，基于提供的基础 CPU 模板，成功实现并优化了一个支持 30 余条指令的五级流水线 CPU。原始模板仅能执行 `ori`、`lui`、`addiu`、`beq` 四条指令，且未处理数据相关与流水线冲突，这构成了我们工作的起点和核心挑战。我们的主要任务是扩展指令集、解决数据和控制冒险，并补全访存逻辑，最终目标是使其能正确执行 `inst` 目录下的所有测试程序。

为高效协作，我们根据代码修改的复杂度和模块职责，将工作量划分为 40%、32% 和 28% 三部分：

- 1) 占 40% 的核心工作集中于 ID 段的改造。此部分涉及从 4 条到 30 余条指令的译码逻辑扩展，并为解决数据冒险，在 `fix/ID.v` 中实现了复杂的 EX、MEM、WB 段数据前递通路，以及 `load-use` 场景下的停顿请求（`stallreq_for_load`），这是确保 `slt`、`mult` 等指令正确执行的关键。
- 2) 占 32% 的工作聚焦于 EX 和 MEM 段的功

能补全，依据 `README.md` 中的访存说明，我们在 `EX.v` 和 `MEM.v` 中添加了对 `sb/sh/sw` 和 `lb/lbu/lh/ld` 等指令的字节/半字对齐、写使能生成和数据扩展逻辑，并实现了除法指令的多周期停顿。

- 3) 剩余 28% 的工作则负责 CTRL 段的全局停顿逻辑实现与 `mycpu_top.v` 的顶层模块集成，确保来自 ID 和 EX 段的停顿请求能被正确仲裁和执行，保证了 CPU 在面对 `start.S` 和 `test.s` 中复杂跳转和异常处理时的稳定性

通过本次实验，我们不仅掌握了 Verilog 硬件描述语言和 Vivado 仿真工具的使用，更在实践中深化了对 CPU 五级流水线、数据前递、流水线停顿等核心概念的理解。每一个 `func_test` 测试点的通过，都是对我们理论知识与工程实践能力的有效验证。这次经历为我们深入学习计算机体系结构。

2. 总体设计

CPU 主体结构遵循经典的五级流水线设计，分为六大部分。其中五部分为流水线的基本组成单元：IF（取指段）、ID（译

码段)、EX (执行段)、MEM (访存段) 和 WB (写回段), 另外一部分为 CTRL (控制段), 负责协调整个流水线的运行。

- 1) IF 段: 其核心功能是根据 PC 的值从指令存储器中获取指令。在设计上, 本段维护着 PC 寄存器, 在每个时钟周期, 它会根据流水线的状态来更新 PC 值。在正常执行时, PC 自动加 4 以指向下一条指令。当流水线中存在分支或跳转指令时, IF 段会采纳 EX 段计算出的目标地址来更新 PC。此外, 为了处理流水线停顿, IF 段会响应来自 CTRL 段的 stall 信号, 当信号有效时, 它将暂停取指并冻结 PC 值, 确保不会取入新的指令, 从而为后续流水段处理冲突争取时间。
- 2) ID 段: 负责对 IF 段传入的指令进行解析, 并为后续阶段准备好所需的操作数和控制信号。在本设计中, 它不仅需要将指令的 opcode、funct 等字段解码为详细的控制信号, 还要从寄存器堆 (regfile.v) 中读取源操作数。同时, 为了解决数据冒险, 本段实现了完整的数据前递 (forwarding) 逻辑, 能够检测 EX、MEM 和 WB 段中正在计算但尚未写回的数据, 并将其直接“前递”给 EX 段的 ALU 输入端。对于无法通过前递解决的“load-use”冒险 (即加载指令的结果紧接着被下一条指令使用), ID 段会主动向 CTRL 段发出停顿请求 (stallreq_for_load), 以插入一个周期的气泡来等待数据就绪。
- 3) EX 段: 负责执行绝大多数指令的计算任务。其核心部件是算术逻辑单元 (alu.v), 它根据 ID 段传递来的控制信号, 对操作数执行指定的算术或逻辑运算。对于访存指令 (如 lw, sw), ALU 被用来计算最终的访存地址; 对于分支指令 (如 beq), ALU 则用来比较操作数以判断分支是否发生。本段还集成了处理乘法和除法 (div.v) 的专用逻辑。由于除法是多周期操作, 在计算期间, EX 段会持续向 CTRL 段发出停顿请求, 直到运算完成, 以防止后续指令

过早进入执行阶段。

- 4) MEM 段: 负责处理与数据存储器的交互。当执行 load 指令时, 本段会根据 EX 段计算出的地址, 向存储器发出读请求并接收返回的数据。当执行 store 指令时, 它会将待写入的数据和地址发送给存储器。设计上, 本段实现了对非对齐访存的精细化处理, 例如, 对于 lb 或 lbu 等字节加载指令, 它能根据地址的最低两位选择正确的字节, 并进行相应的符号位或零扩展。对于非访存指令, MEM 段则相当于一个“直通”阶段, 仅将 EX 段的运算结果传递给 WB 段。
- 5) WB 段: 负责将指令的最终执行结果写回到寄存器堆中。本段的逻辑相对简单, 它会根据指令类型, 通过一个选择器决定最终要写回的数据来源——这个数据可能来自 MEM 段从存储器中读取的值, 也可能来自 EX 段的 ALU 运算结果。在时钟的上升沿, 选定的数据被写入指令所指定的目标寄存器 (rd 或 rt), 从而完成一条指令的执行周期。
- 6) CTRL 段: 负责通过生成全局控制信号来协调各流水段的同步与暂停。作为一个中央仲裁单元, 它接收来自 ID 段的 stallreq_for_load 信号和来自 EX 段的多周期指令 (如除法) 停顿请求。当任一请求有效时, 它会生成一个多位的 stall 总线信号, 该信号被分发到 IF、ID、EX 等段, 能够精确地控制哪些阶段需要暂停, 哪些阶段需要插入气泡, 从而确保流水线在遇到数据冒险或结构冒险时能够正确、有序地运行, 避免数据错误。

3. 实验设计与实现

3.1. IF 段

3.1.1. 设计目标

IF 段完成取指地址 PC 的产生与更新、向指令 SRAM 发起读请求, 并将取指得到的 PC (以及取指使能) 通过 IF/ID 总线传递给 ID 段。支持:

- ①顺序执行: $PC \leftarrow PC + 4$
 ②分支/跳转: $PC \leftarrow br_addr$
 ③暂停: 当 $stall[0] == Stop$ 时冻结 PC 与取指使能

3.1.2. 接口设计

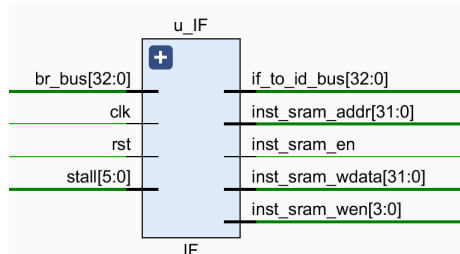


图 1 接口示意图

1. 输入信号:

(1) clk: 时钟信号, 所有寄存器 (如 PC) 都在时钟上升沿更新。

(2) rst: 复位信号, 高电平有效, 复位时 PC 会被置为初始地址 0xbfbf_fffc

(3) stall: 暂停控制信号, 这里主要用到 $stall[0]$, 如果 $stall[0] == 1$, PC 寄存器就不更新 (保持原值), 相当于“冻结”当前取指操作。

(4) br_bus: 分支跳转总线, 决定下一条指令是从 $PC+4$ 取, 还是跳到别的地方去, br_e: 分支使能。如果为 1, 说明要发生跳转, 否则: $pc_reg + 4$
br_addr: 跳转的目标地址

2. 输出信号:

(1) 发给 ID 模块:

if_to_id_bus: 把当前的 PC 值传给 ID 阶段

(2) 发给指令存储器:

①inst_sram_en: 告诉 SRAM: “我要开始工作了, 请准备好。” (代码中用 ce_reg 控制, 复位后置 1)

②inst_sram_wen: 写使能信号, 因为 IF 阶段只读指令, 不修改指令, 所以这里恒为 4'b0

③inst_sram_addr: 地址信号, 告诉 SRAM: “我要读在这个地址的指令。” (直接连着 pc_reg)

④inst_sram_wdata: 写数据信号, 因为是只读, 所以这个信号没用, 恒为 32'b0

3.2. ID 段

3.2.1. 整体功能

译码阶段 (ID Stage) 是流水线的控制中枢, 主要承担指令译码、寄存器堆读取、操作数生成、分支判断以及流水线冒险检测等关键任务。在本实验中, 基于基础模板, 对 ID 模块进行了深度的功能扩展与逻辑重构, 使其能够支持完整的 MIPS 指令集运算及高效的流水线冲突处理。

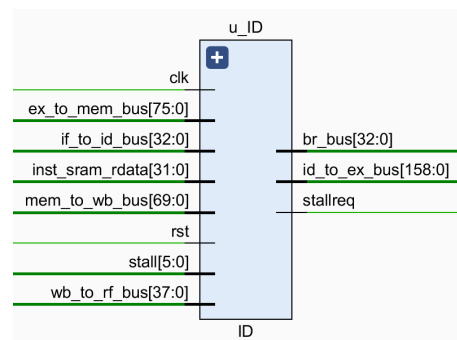


图 2 接口示意图

3.2.2. 指令集架构的完备化扩展

GitHub 上的原始设计文件仅支持 ori, lui, addiu, beq 四条基础指令。为了满足测试点需求, 在 ID 级完成了对 MIPS32 核心指令集的全面扩充。

(1) 算术与逻辑指令扩展: 增加了 add/addu, sub/subu, and, or, xor, nor, slt/sltu 等 R 型指令的译码逻辑, 并完善了 andi, xori, slti 等 I 型指令的立即数扩展 (符号扩展/零扩展) 选择逻辑。

(2) 移位指令支持: 新增了 sll, srl, sra 及其变体指令的支持, 并在 ID 级完成了移位量 sa 的选通控制。

(3) 乘除法与 HILO 寄存器支持: 针对 mult, div 等多周期指令及 mfhi, mflo, mthi, mtlo 数据移动指令, 在 ID 级增加了相应的译码信号, 确保能正确控制 EX 级的乘除法单元。

(4) 访存与分支指令: 完善了 lb, lhu, sb, sh 等非对齐访存指令的控制信号生成; 扩充了 jal, jr, bne, bgez 等复杂的跳转与分支指令。

通过模块化调用 decoder_6_64 和 decoder_5_32, 实现了操作码 (Opcode) 与功能码 (Function Code) 的双层译码, 生成了 alu_op、mem_op 及 rf_we 等关键控

制信号总线。

3.2.3. 数据相关性处理与前推机制

这是 ID 阶段最核心的设计难点。为了解决流水线中的“写后读”（RAW）数据冒险，避免不必要的流水线暂停，在 ID 阶段引入了全功能的内部数据前推机制。

(1) 冲突检测逻辑：设计了比较电路，实时监测当前指令的源寄存器地址(rs, rt)与后续流水级（EX 级、MEM 级）的目标写入寄存器地址(ex_waddr, mem_waddr)是否发生冲突。其逻辑表达式为 $rs_ex_ok = (rs == ex_waddr) \&\& ex_we;$

(2) 多级前推网络：基于冲突检测结果，实现了多路选择器逻辑。系统优先选择 EX 级的数据，其次选择 MEM 级的数据，最后才选择寄存器堆读取的数据。生成的 rdata1_fd 和 rdata2_fd 信号替代了原始的寄存器读出值，确保了发送给 EX 级 ALU 的操作数始终是正确的，极大提高了流水线效率。

3.2.4. Load-Use 冒险检测与流水线暂停

针对前推机制无法解决的 Load-Use 冒险，即上一条指令为 Load 指令，且其结果被当前 ID 级指令立即使用的情况，在 ID 级设计了专门的检测逻辑：

(1) 检测机制：通过解析 EX 级传回的 ex_ram_read 信号判断上一条指令是否为访存读取操作，并结合寄存器地址冲突检测。

(2) 暂停请求生成：当满足 $ex_ram_read \& (rs_ex_ok \mid rt_ex_ok)$ 条件时，ID 模块拉高 stallreq_for_load 信号。该信号直接作用于全局控制模块，冻结 PC 与 ID 级寄存器，并在 EX 级插入气泡，从而保证数据流的正确性。

3.2.5. 提前分支决断

为了减少分支预测失败带来的流水线代价，我将分支跳转的判断逻辑从 EX 阶段提前至 ID 阶段完成。

(1) 逻辑实现：利用前推后的操作数 rdata1_fd 和 rdata2_fd 在 ID 级直接进行相等 (beq)、不等 (bne)、小于零 (bltz) 等条件判断。

(2) 跳转地址计算：在 ID 级集成了加法器逻辑，根据当前 PC 值与立即数偏移量

并行计算跳转目标地址 br_addr。

(3) 性能提升：该设计将分支延迟槽严格控制 1 个时钟周期，结合延迟槽技术，有效避免了控制冒险带来的性能损失。

3.3. EX 段

3.3.1. 整体功能

EX 段是 CPU 五级流水线的核心计算阶段。它接收从 ID 段传递来的指令信息和操作数，负责完成指令指定的核心任务。

其首先是利用 ALU 来执行加、减、与、或、移位等各类算术逻辑运算，并为 load/store 指令计算出最终的有效访存地址。此外，EX 段还需处理更为复杂的指令：它不仅管理乘法、除法等需要多个时钟周期才能完成的多周期指令，并在其执行期间向 CTRL 发出停顿请求以暂停前级流水线；还要负责对 HI / LO 特殊寄存器进行读写操作，以支持这些乘除法以及 mthi / mtlo / mfhi / mflo 等相关指令。针对非字对齐的存储指令，EX 段还能生成精确的字节写使能信号和对齐后的写入数据，确保数据精确写入。最终，EX 段会将运算结果、访存地址以及所有相关的控制信号打包，通过 ex_to_mem_bus 总线传递给下一级流水线 MEM 段。

3.3.2. ALU 运算单元

ALU 是 EX 段的计算核心，负责执行大部分算术和逻辑指令。

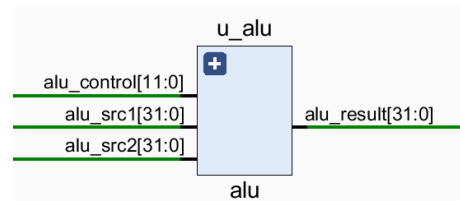


图 3 alu 模块接口

其工作流程如下：首先根据指令类型，通过多路选择器动态选择其两个操作数（alu_src1 和 alu_src2），这些操作数可能来自通用寄存器、指令中的立即数或 PC 值；

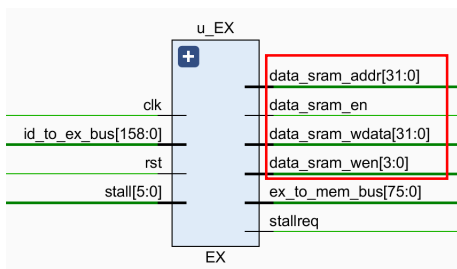
```
//操作数选择
assign alu_src1 = sel_alu_src1[1] ? ex_pc :
                  sel_alu_src1[2] ? sa_zero_extend : rf_rdata1;

assign alu_src2 = sel_alu_src2[1] ? imm_sign_extend :
                  sel_alu_src2[2] ? 32'd8 :
                  sel_alu_src2[3] ? imm_zero_extend : rf_rdata2;
```

随后，这两个操作数与 ID 段传来的 `alu_op` 运算控制码一同送入 `u_alu` 模块，执行加、减、逻辑运算等操作，并输出 32 位的运算结果 `alu_result`，这个结果是本阶段大多数指令的执行结果。

3.3.3. 内存访问逻辑

该部分逻辑专为 `store` 指令设计，其核心目标是根据访存地址，生成精确的字节写使能 (`data_sram_wen`) 和对齐后的写入数据 (`data_sram_wdata`)。



它首先提取地址最低两位作为偏移量，然后在一个组合逻辑块中，根据指令类型和偏移量，计算出应使能哪些字节以及如何排布待写数据，最后将这些信号输出给下一级流水线和 SRAM。

3.3.4. 多周期指令与停顿请求

该部分主要用于处理执行时间超过一个周期的除法指令，确保在计算完成前，流水线保持静止。当检测到 `div` 或 `divu` 指令时，它会启动 `div` 运算单元。在除法器计算期间，模块会持续发出一个停顿请求给顶层 CTRL 模块，以冻结前级流水线。一旦计算完成，停顿请求即被撤销，流水线恢复正常流动，从而确保了在得到除法结果前不会执行后续指令。

3.3.5. HI/LO 特殊寄存器管理

HI / LO 寄存器管理逻辑分为“写”和“读”两部分。写操作在一个时序逻辑块中完成，它根据优先级更新寄存器：首先是锁存除法结果；

```
// --- HI/LO 寄存器写逻辑 ---
always @(posedge clk) begin
    if (rst) begin
        hi <= 32'b0;
        lo <= 32'b0;
    end
    // 当除法单元计算完成时，更新HI/LO
    else if (div_ready_i == `DivResultReady) begin
```

图 4 HI/LO 写操作

其次是执行 `mthi / mtlo` 指令，最后是

锁存乘法结果。读操作则在一个组合逻辑中实现，当指令为 `mfhi / mflo` 时，将 HI / LO 寄存器的值作为最终的执行结果 `ex_result` 输出。

```
// --- 结果选择逻辑 ---
assign ex_result = inst_mfhi ? hi :
// 如果是mfhi，结果是hi寄存器的值
inst_mflo ? lo :
// 如果是mflo，结果是lo寄存器的值
alu_result;
// 否则，结果是ALU的运算结果
```

图 5 读操作

3.4. MEM 段

MEM 段是流水线中专门处理与内存交互的阶段。其功能相对集中，主要负责处理 `load` 指令的数据读取和 `store` 指令的数据写入。它接收从 EX 段传来的访存地址和控制信号，如果是 `load` 指令，它会接收来自 SRAM 的 32 位数据 (`data_sram_rdata`)，并根据具体的指令对该数据进行精确的字节/半字选择和符号/零扩展。最终，它将处理后的数据或从 EX 段透传的 ALU 结果打包，通过 `mem_to_wb_bus` 总线传递给流水线的最后一级——WB 段。

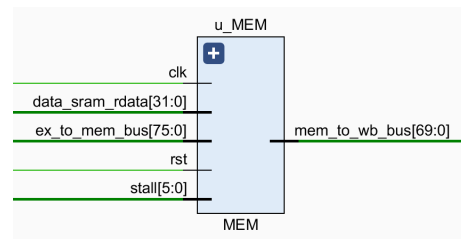


图 6 接口示意图

3.4.1. 内存读取结果处理

该部分是 MEM 段的核心，专用于处理 `load` 指令。它根据指令类型 (`lw`, `lb`, `lh` 等)，从 SRAM 读出的 `data_sram_rdata` 中，精确地选择出目标字节或半字。接着，它会根据指令要求进行符号位扩展或零扩展。最后，通过一个选择器，决定最终是把这个处理过的内存数据，还是 EX 段的 ALU 运算结果，作为写回数据传递给下一级。

```
// 从ex_to_mem_bus_r总线中的mem_op字段解析出具体的load指令类型
wire inst_lb,inst_lbu,inst_lh,inst_lhu,inst_lw;
assign {
    inst_lb,
    inst_lbu,
    inst_lh,
    inst_lhu,
    inst_lw
} = mem_op;
```


3.4.2. 数据输出

MEM 段将最终确定的写回数据、目标寄存器地址和写使能信号打包到 mem_to_wb_bus 总线上。这个总线作为模块的主要输出，将所有写回操作所需的信息一次性传递给下一级 WB 段，由其在下个周期完成最终的寄存器写入，清晰地划分了访存与写回的职责。

```
// 将所有写回操作所需的信息打包到mem_to_wb_bus总线，传递给WB段
assign mem_to_wb_bus = {
    mem_pc,      // 69:38
    rf_we,       // 37
    rf_waddr,    // 36:32
    rf_wdata     // 31:0
};
```

3.5. WB 段

3.5.1. 设计目标

WB 段接收 MEM/WB 总线中的“写回信息”，在时钟沿锁存形成流水寄存器，然后输出：

- 1) 写回寄存器堆的控制信号与数据 (wb_to_rf_bus)
- 2) 课程要求的调试接口 (debug_wb_*) —— 用于与参考模型对拍

3.5.2. 接口设计

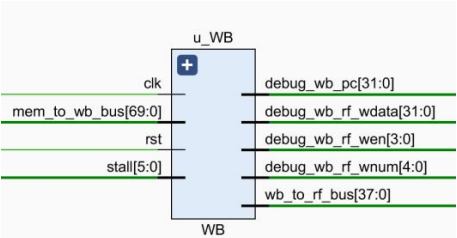


图 7 接口示意图

1. 输入信号：

- (1) clk: 时钟信号
- (2) rst: 复位信号
- (3) stall: 暂停控制信号，这里主要用到 stall[4] 和 stall[5]。用于控制 MEM 到 WB 的流水线寄存器更新
- (4) mem_to_wb_bus: MEM 到 WB 数据总线，这是上一级 (MEM) 传过来的包裹，里面包含了所有 WB 阶段需要的信息：
 - ①wb_pc: 当前指令的 PC
 - ②rf_we: 寄存器写使能，为 1 表示这条指令要写寄存器。
 - ③rf_waddr: 要写的寄存器编号
 - ④rf_wdata: 要写进去的数据 (可能是 ALU

的计算结果，也可能是从 RAM 读出来的数据)

2. 输出信号：

- (1) 回写给 ID 模块
 - wb_to_rf_bus: 写回总线，包含 {rf_we, rf_waddr, rf_wdata}。这条线会一路拉回到 ID 模块 (因为寄存器堆在 ID 模块里)。当 ID 模块在时钟上升沿检测到这个信号时，就会把 rf_wdata 写入到 rf_waddr 指定的寄存器中。
- (2) 调试信号
 - ①debug_wb_pc: 当前写回指令的 PC。告诉评测机：“我现在执行完了这条指令。”
 - ②debug_wb_rf_wen: 写使能
 - ③debug_wb_rf_wnum: 写的目标寄存器号，告诉评测机：“我改了第几号寄存器。”
 - ④debug_wb_rf_wdata: 写入的数据，告诉评测机：“我把这个寄存器改成了多少。”

3.6. CTRL 段

3.6.1. 设计目标

CTRL 段负责根据流水线各级发出的暂停请求生成全局 stall[5:0] 信号，对各级流水寄存器/PC 的更新进行统一控制，从而解决：

- (1) Load-use 数据相关导致的 ID 阶段需要等待的需求
- (2) EX 阶段多周期或其它原因导致的全流水线暂停

本设计采用 6 位 stall 总线，并规定 Stop=1、NoStop=0。

3.6.2. 接口设计

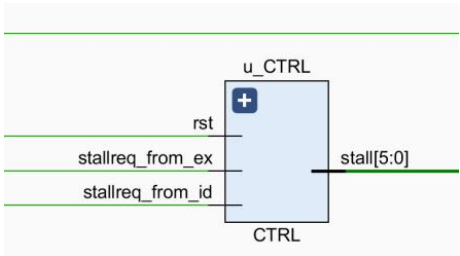


图 8 接口示意图

- (1) rst: 复位信号
- (2) stallreq_for_load: 来自 ID 阶段的请求 (Load 冲突)
- (3) stallreq_from_ex: 来自 EX 阶段的

请求

(4) 输出: reg [`StallBus-1:0] stall:
输出给所有流水线寄存器的控制信号(6 位)

流水线寄存器	控制信号
stall[0]	控制 PC 更新 (取指阶段的源头)
stall[1]	控制 IF/ID 寄存器 (取指 -> 译码)
stall[2]	控制 ID/EX 寄存器 (译码 -> 执行)
stall[3]	控制 EX/MEM 寄存器 (执行 -> 访存)
stall[4]	控制 MEM/WB 寄存器 (访存 -> 写回)
stall[5]	控制 WB 后的状态 (目前代码中主要用于配合 stall[4] 产生气泡)

3.6.3. 暂停策略

CTRL 段采用组合逻辑，优先级为：stallreq_from_ex 高于 stallreq_for_load (因为 EX 阶段暂停通常影响更深流水级)。实现如下 (见 CTRL.v)：

(1) 复位：stall = 6'b000000 (全不暂停)

(2) EX 请求暂停：stall = 6'b001111 表示暂停 PC/IF/ID/EX (从低位到高位对应前级到后级，配合各级对 stall[i] 的使用实现冻结)

乘法是在 EX 阶段算的，所以 EX 必须停住 (stall[3]=1)，前面的 ID, IF, PC 也要停。但是，已经在 MEM 和 WB 阶段的指令 (比如前一条指令是加法，已经算完了正在写回)，它们没必要陪着 EX 阶段一起等，它们应该继续跑完。所以 stall[4] 和 stall[5] 是 0

(3) Load-use 请求暂停：stall = 6'b000111 表示暂停 PC/IF/ID，让 EX/MEM/WB 继续推进，从而在下一拍获得可用数据，解除相关

只需要停住 ID 阶段去等数据，后面的 EX, MEM, WB 都可以继续跑完，所以不需要停 stall[3/4/5]

3.6.4. 与流水线其它级的配合说明

1) IF 段通过 stall[0] 冻结 PC

2) ID 段通过 stall[1] 控制 IF/ID 流水寄存器是否更新，并在“本级暂停但下级不暂停”时注入气泡 (ID.v 的 stall[1]==Stop && stall[2]==NoStop 分支)

3) MEM/WB 段同理在“本级暂停、下级不暂停”时清空寄存器形成气泡 (WB.v 的 stall[4]==Stop && stall[5]==NoStop 分支)

4. 团队协作与分工

4.1. 问题解决

在本次实验中，我们遇到了多个需要跨模块、跨职能协作解决的复杂问题。通过紧密的团队沟通与分工合作，我们成功克服了这些挑战，确保了 CPU 流水线的正确性和高效性。以下为几个典型问题：

1. 问题一：Load-Use 数据冒险的解决

➤ 描述：

当一条加载指令 (如 lw) 之后紧跟着一 条使用其加载结果的指令时，会产生 “Load-Use” 数据冒险。若不处理，后一条指令会读到旧的、错误的数据。

➤ 解决方案与协作：

高荧首先在 ID.v 中设计了冒险检测逻辑。当译码发现当前指令的源寄存器与上 一条 lw 指令的目标寄存器匹配时，立即向 控制段发出 stallreq_for_load 停顿请求。

贾培鑫在 CTRL.v 中接收此请求，并将 其整合到全局停顿策略中，生成 stall 总线 信号，精确地冻结 PC 和 IF/ID 寄存器，同 时向 EX 段注入一个“气泡”。

➤ 团队协作：

高荧和贾培鑫共同定义了停顿请求信 号的接口与时序。高荧负责“发现问题”， 贾培鑫负责“解决问题”，二者协作确保了 ID 段的局部请求能被 CTRL 段准确地转化 为全局流水线行为，从而解决了该数据冒险。

2. 问题二：数据前递通路的建立与实现

➤ 描述：

为了避免不必要的停顿、提升流水线效率，绝大多数数据冒险需要通过数据前递机制解决。这要求将 EX、MEM 段的计算结果直接“旁路”回 ID 段供后续指令使用，涉及多个模块的协同改造。

➤ 解决方案与协作：

贾培鑫首先在 mycpu_core.v 中进行“结构改造”，将后级流水线总线（ex_to_mem_bus，mem_to_wb_bus）从输出端重新布线，引回到 ID 段的输入端，为数据前递建立了物理通路。

高荧接着在 ID.v 中增加了复杂的比较和选择逻辑。它实时比较当前指令的源寄存器与 EX、MEM 段正在处理的指令的目标寄存器，若匹配则生成控制信号，选择从旁路传回的数据，而非寄存器堆的旧数据。

张诗琪负责确保其输出总线上的数据（如 ALU 计算结果、访存读出数据）和控制信号（如写回寄存器地址）的正确性，为 ID 段的决策逻辑提供了可靠的数据源。

➤ 团队协作：

这是一个典型的“搭台唱戏”式合作。贾培鑫负责“搭台”（布线），高荧负责“唱戏”（实现核心判断逻辑），张诗琪则保证了“戏”所需要的数据准确无误。三方通过 defines.vh 中定义的总线结构紧密合作，最终实现了高效的数据前递。

3. 问题三：多周期除法指令导致的结构冒险

➤ 描述：

div 指令在 EX 段需要多个时钟周期才能完成。在此期间，必须阻止后续指令进入 EX 段，否则会产生结构冒险（即多个指令争用同一个硬件资源）。

➤ 解决方案与协作：

张诗琪在 EX.v 中为除法单元设计了一个状态机。在除法运算的多个周期内，该状态机持续输出 stallreq_from_ex 停顿请求信号。

贾培鑫在 CTRL.v 中增加了对 stallreq_from_ex 信号的处理逻辑。一旦检测到该信号，立即暂停流水线的前三个阶段（IF，ID，EX 的输入），直到 EX 段完成运算并撤销该请求。

➤ 团队协作：

张诗琪和贾培鑫再次展现了“请求-响应”的协作模式。张诗琪作为“请求方”，在数据通路内部根据运算状态发出请求；贾培鑫作为“响应方”，将该局部请求转化为全局的流水线控制行为。这种分层设计使得 EX 段可以专注于运算逻辑，而 CTRL 段则统一管理流水线调度，体现了清晰的设计分工。

4.2. 实验心得

4.2.1. 高荧

本次五级流水线 CPU 的设计实验对我而言，是一次从理论认知到工程实践的深度跨越。在实验初期，面对复杂的 MIPS 指令集架构和庞大的信号互连，我曾一度感到无从下手。为了克服这一困难，我并没有急于编写具体的逻辑代码，而是首先沉下心来梳理数据通路，确立了基于总线（Bus）的数据传递策略。通过将 IF、ID、EX、MEM、WB 各阶段所需的控制信号和数据打包成结构化的总线接口，我成功地将复杂的顶层连线简化，这不仅降低了模块间的耦合度，也为后续的排错和功能扩展打下了坚实的基础。这一过程让我深刻体会到，良好的系统架构设计是应对复杂硬件工程的关键。

在具体功能的实现过程中，最让我头疼但也收获最大的挑战在于流水线冒险的处理。在实现数据前推机制时，我最初很难理清不同流水级之间数据的时序关系，导致经常出现读取旧值的情况。为了解决这个问题，我绘制了详细的时序图，仔细分析了 EX 级和 MEM 级数据回传的路径，最终在 ID 阶段通过多路选择器实现了优先级的判定，确保了 ALU 始终能获取最新的操作数。此外，Load-Use 冒险的处理也让我印象深刻，单纯的前推无法解决数据未就绪的问题，我通过在 ID 级增加检测逻辑，精准地发出 stallreq 信号来冻结流水线。这一过程让我明白了硬件设计不能仅考虑理想情况，更要通过严密的逻辑去处理各种边缘冲突。

除了了解冲突，复杂指令的实现也极大地锻炼了我的逻辑思维能力。特别是针对非对齐访存指令（如 sb、sh）和多周期除法

指令的设计，打破了我对单周期操作的固有思维。在处理非对齐存储时，我需要根据地址的低两位精细控制 SRAM 的写使能信号，这要求我对数据的字节序和位操作有极高的敏感度。而在实现除法器时，为了适配流水线的节奏，我引入了握手协议，通过暂停流水线来等待运算完成。这些细节的打磨，让我意识到 CPU 设计不仅仅是堆砌逻辑门，更是在性能、面积和正确性之间做出的不断权衡。

最后，调试过程是整个实验中最煎熬但也最令人兴奋的环节。从最初面对波形图中满屏的蓝色“Z”态（高阻）不知所措，到后来学会顺藤摸瓜，逐级排查信号的驱动源；从面对测试点报错时的焦虑，到静下心来通过反汇编代码定位到具体的移位指令或跳转逻辑错误。每一次波形图的分析，都是对自己逻辑漏洞的一次修补。当控制台最终连续打印出 64 个“PASS”时，那种成就感无以言表。这次实验不仅让我掌握了 Verilog 硬件描述语言和 MIPS 架构的精髓，更重要的是，它培养了我严谨的工程素养和面对复杂 Bug 时冷静分析、抽丝剥茧解决问题的能力。这将是未来计算机系统学习道路上最宝贵的财富。

4.2.2. 贾培鑫

在本次实验中，我承担了 CTRL（控制单元）的设计与 mycpu_core（顶层核心）的集成工作。

在 CTRL 模块的开发中，我面临的最大挑战是如何精准地控制流水线的节奏。起初，我以为“暂停”就是简单的让整个 CPU 停下来，但真正上手后才发现，流水线暂停还是很难的。

为了解决 Load-Use 冒险，我不能粗暴地拉停所有阶段，而是必须实现“前停后跑”——冻结 PC、IF 和 ID 段，同时让 EX 及之后的阶段继续运行以取回内存数据。这种逻辑在代码中体现为 `stall = 6'b000111`，这一行简单的赋值背后，是我对“气泡（Bubble）”概念的深刻理解：暂停本质上是在流水线中插入空隙，用时间换取数据的正确性。

而在处理乘除法这种多周期指令时，我

又必须调整策略，利用 `stallreq_from_ex` 信号将 EX 及其之前的阶段全部锁死。通过在 `always` 块中构建严格的 if-else 优先级判断（复位 > EX 请求 > ID 请求），我学会了如何在硬件层面仲裁冲突，确保 CPU 在面对复杂工况时依然井然有序。

`mycpu_core` 的编写则是对耐心和全局观的考验。作为顶层文件，它不仅是所有子模块的容器，更是数据流动的枢纽。

我需要将队友编写的 IF、ID、EX、MEM、WB 模块一个个实例化，并像连接神经网络一样，把成百上千根信号线准确无误地对接起来。在这个过程中，我重点打通了反馈通路：将 ID 段的 `stallreq_for_load` 和 EX 段的 `stallreq` 信号一路牵引至 CTRL 模块，再将 CTRL 生成的 `stall` 信号广播回各个流水线寄存器。这让我深刻体会到了“系统集成”的含义——单个模块功能的正确并不代表 CPU 能跑通，只有当模块间的握手信号（Handshake）严丝合缝时，静态的代码才能变成动态流转的指令流。

总的来说，这次实验让我跳出了单一指令的视角，开始站在系统架构的高度思考问题。从最初面对一堆离散模块的手足无措，到最终看到波形图中 `stall` 信号随着指令流精准起落，那种成就感，是任何理论课都无法替代的。

4.2.3. 张诗琪

在本次计算机系统实验中，我主要负责 EX 和 MEM 这两个核心数据通路阶段的开发。回顾整个过程，我深刻体会到，将课本上抽象的流水线概念转化为具体、严谨的 Verilog 代码，是一项极具挑战也极富收获的任务。我的工作为 CPU 构建起处理复杂运算和精确访存的关键能力。

我的首要任务是在 EX 段实现访存写指令。为了支持 `sb`、`sh` 和 `sw`，我必须精确生成 4 位的字节写使能信号 `data_sram_wen`。这让我第一次在硬件层面直观地理解了字节编址的含义：`sw` 对应 `4'b1111`，而 `sb` 和 `sh` 则需要根据地址的最低两位来动态选择写入的字节位置。此外，实现乘除法功能也远非调用一个模块那么简单。我添加了 HI 和 LO 寄存器，并设计了 `mult/div` 指令对其

进行更新、mfhi/mflo 等指令对其进行读写的完整通路。这项工作的前提是 ID 段能够精确解析这些新增的指令并生成正确的控制信号，同时 CTRL 段必须能正确响应我从 EX 段发出的停顿请求，这让我深刻理解了各流水线阶段环环相扣的精密关系。

在 MEM 段，我面临的挑战是如何处理加载指令。从 SRAM 读出的数据始终是 32 位的，但对于 lb、lbu 等指令，我们需要的只是其中的一部分。因此，我必须在 MEM.v 中根据地址的低位，从读出的 32 位数据中正确选择出目标字节或半字。更关键的是，我需要根据指令是带符号加载，还是无符号加载，来实现对应的符号位扩展或零扩展逻辑，最终将数据正确地扩展为 32 位再送往 WB 段。这个过程不仅锻炼了我的逻辑设计能力，更让我对计算机中数据表示和类型转换的底层原理有了透彻的理解。

最后，仿真调试是整个实验中最困难却也最有成就感的阶段。初期面对仿真，我最困惑的问题是发现 PC 值在波形图上“倒退”，即修改某些代码部分后所得结果与预期完全不符。在反复摸索后，我逐渐学会了如何高效地分析流水线波形图，不再是无头苍蝇。我开始重点追踪几个关键信号：如 debug_wb_pc 与 ref_wb_pc 的对比，各级流水线的 PC 值，用以定位 PC 发生异常跳转的具体阶段；ID 段解析出的指令码 inst，用以确认是否是跳转指令本身逻辑错误；以及 CTRL 段发出的 stall 信号，用以判断是否是错误的流水线控制导致了 PC 被异常刷新。通过这种方法，我最终定位到是导致了 PC “倒退”的原因。每一次这样的“破案”，都是对自己逻辑漏洞的一次精准修复。当最终前 64 个测试点都显示“PASS”时，兴奋是难以言喻。

总而言之，通过亲手搭建 EX 和 MEM 这两个数据通路中最为“硬核”的部分，我将数据前递、流水线停顿、内存对齐等理论知识内化为了实践技能。从空无一物的模板文件到能够正确处理复杂访存和多周期运算的精密逻辑，这个从无到有的过程，是我本次实验最大的收获。

5. 部分实验结果展示

```
run all

Test begin!
----[ 14025 ns] Number 8'd01 Functional Test Point PASS!!!
[ 22000 ns] Test is running, debug_wb_pc = 0xbfc5e4d4
[ 32000 ns] Test is running, debug_wb_pc = 0xbfc5f474
----[ 40475 ns] Number 8'd02 Functional Test Point PASS!!!
[ 42000 ns] Test is running, debug_wb_pc = 0xbfc69440
----[ 49355 ns] Number 8'd03 Functional Test Point PASS!!!
[ 52000 ns] Test is running, debug_wb_pc = 0xbfc3a5d8
[ 62000 ns] Test is running, debug_wb_pc = 0xbfc3a260
----[ 71115 ns] Number 8'd04 Functional Test Point PASS!!!
[ 72000 ns] Test is running, debug_wb_pc = 0xbfc23898
[ 82000 ns] Test is running, debug_wb_pc = 0xbfc24c40
[ 92000 ns] Test is running, debug_wb_pc = 0xbfc2621c
[ 102000 ns] Test is running, debug_wb_pc = 0xbfc2776c
----[ 104845 ns] Number 8'd05 Functional Test Point PASS!!!
[ 112000 ns] Test is running, debug_wb_pc = 0xbfc4a0ac
----[ 117885 ns] Number 8'd06 Functional Test Point PASS!!!
[ 122000 ns] Test is running, debug_wb_pc = 0xbfc6a68c
[ 132000 ns] Test is running, debug_wb_pc = 0xbfc6b62c
[ 142000 ns] Test is running, debug_wb_pc = 0xbfc6c5cc
----[ 144265 ns] Number 8'd07 Functional Test Point PASS!!!
[ 152000 ns] Test is running, debug_wb_pc = 0xbfc509e4
[ 162000 ns] Test is running, debug_wb_pc = 0xbfc51984
----[ 167675 ns] Number 8'd08 Functional Test Point PASS!!!
[ 172000 ns] Test is running, debug_wb_pc = 0xbfc03bb0
[ 182000 ns] Test is running, debug_wb_pc = 0xbfc04b50
----[ 185575 ns] Number 8'd09 Functional Test Point PASS!!!
[ 192000 ns] Test is running, debug_wb_pc = 0xbfc3e008
[ 202000 ns] Test is running, debug_wb_pc = 0xbfc3efa8
----[ 203475 ns] Number 8'd10 Functional Test Point PASS!!!
[ 212000 ns] Test is running, debug_wb_pc = 0xbfc6f800
[ 222000 ns] Test is running, debug_wb_pc = 0xbfc707a0
----[ 222735 ns] Number 8'd11 Functional Test Point PASS!!!
[ 232000 ns] Test is running, debug_wb_pc = 0xbfc02648
----[ 237365 ns] Number 8'd12 Functional Test Point PASS!!!
[ 242000 ns] Test is running, debug_wb_pc = 0xbfc3faec
[ 252000 ns] Test is running, debug_wb_pc = 0xbfc40f3c

----[ 499435 ns] Number 8'd27 Functional Test Point PASS!!!
[ 502000 ns] Test is running, debug_wb_pc = 0xbfc8a360
[ 512000 ns] Test is running, debug_wb_pc = 0xbfc8b300
[ 522000 ns] Test is running, debug_wb_pc = 0xbfc8c2a0
----[ 525825 ns] Number 8'd28 Functional Test Point PASS!!!
[ 532000 ns] Test is running, debug_wb_pc = 0xbfc78914
[ 542000 ns] Test is running, debug_wb_pc = 0xbfc798b4
----[ 546225 ns] Number 8'd29 Functional Test Point PASS!!!
[ 552000 ns] Test is running, debug_wb_pc = 0xbfc475a4
[ 562000 ns] Test is running, debug_wb_pc = 0xbfc48544
[ 572000 ns] Test is running, debug_wb_pc = 0xbfc494e4
----[ 572635 ns] Number 8'd30 Functional Test Point PASS!!!
[ 582000 ns] Test is running, debug_wb_pc = 0xbfc09260
[ 592000 ns] Test is running, debug_wb_pc = 0xbfc0a200
----[ 593035 ns] Number 8'd31 Functional Test Point PASS!!!
[ 602000 ns] Test is running, debug_wb_pc = 0xbfc7b40
[ 612000 ns] Test is running, debug_wb_pc = 0xbfc77ae0
----[ 615185 ns] Number 8'd32 Functional Test Point PASS!!!
[ 622000 ns] Test is running, debug_wb_pc = 0xbfc42d9c
[ 632000 ns] Test is running, debug_wb_pc = 0xbfc43d3c
----[ 634365 ns] Number 8'd33 Functional Test Point PASS!!!
[ 642000 ns] Test is running, debug_wb_pc = 0xbfc0d4ac
[ 652000 ns] Test is running, debug_wb_pc = 0xbfc0e44c
----[ 656735 ns] Number 8'd34 Functional Test Point PASS!!!
[ 662000 ns] Test is running, debug_wb_pc = 0xbfc0eb28
[ 672000 ns] Test is running, debug_wb_pc = 0xbfc07ac8
----[ 676065 ns] Number 8'd35 Functional Test Point PASS!!!
[ 682000 ns] Test is running, debug_wb_pc = 0xbfc5bc14
[ 692000 ns] Test is running, debug_wb_pc = 0xbfc5cbb4
----[ 698445 ns] Number 8'd36 Functional Test Point PASS!!!
[ 702000 ns] Test is running, debug_wb_pc = 0xbfc595fc
[ 712000 ns] Test is running, debug_wb_pc = 0xbfc57bfc
[ 722000 ns] Test is running, debug_wb_pc = 0xbfc59250
[ 732000 ns] Test is running, debug_wb_pc = 0xbfc5a8d4
----[ 736645 ns] Number 8'd37 Functional Test Point PASS!!!
[ 742000 ns] Test is running, debug_wb_pc = 0xbfc1eae8
[ 752000 ns] Test is running, debug_wb_pc = 0xbfc20170
[ 762000 ns] Test is running, debug_wb_pc = 0xbfc217e4
[ 772000 ns] Test is running, debug_wb_pc = 0xbfc22e94

----[1185635 ns] Number 8'd48 Functional Test Point PASS!!!
[1192000 ns] Test is running, debug_wb_pc = 0xbfc17f80
----[1195145 ns] Number 8'd49 Functional Test Point PASS!!!
[1202000 ns] Test is running, debug_wb_pc = 0xbfc60e04
----[1204035 ns] Number 8'd50 Functional Test Point PASS!!!
[1212000 ns] Test is running, debug_wb_pc = 0xbfc03b7c
----[1215135 ns] Number 8'd52 Functional Test Point PASS!!!
[1221745 ns] Number 8'd53 Functional Test Point PASS!!!
[1222000 ns] Test is running, debug_wb_pc = 0xbfc00d94
----[1228035 ns] Number 8'd54 Functional Test Point PASS!!!
[1232000 ns] Test is running, debug_wb_pc = 0x00000000
----[1234645 ns] Number 8'd55 Functional Test Point PASS!!!
[1241885 ns] Number 8'd56 Functional Test Point PASS!!!
[1242000 ns] Test is running, debug_wb_pc = 0xbfc0d60
----[1248485 ns] Number 8'd57 Functional Test Point PASS!!!
[1252000 ns] Test is running, debug_wb_pc = 0xbfc4fa0c
----[1253335 ns] Number 8'd58 Functional Test Point PASS!!!
[1262000 ns] Test is running, debug_wb_pc = 0xbfc37e08
[1272000 ns] Test is running, debug_wb_pc = 0xbfc38d3c
----[1277255 ns] Number 8'd59 Functional Test Point PASS!!!
[1282000 ns] Test is running, debug_wb_pc = 0xbfc67fc4
[1292000 ns] Test is running, debug_wb_pc = 0xbfc68e0
[1302000 ns] Test is running, debug_wb_pc = 0xbfc69e00
----[1302835 ns] Number 8'd60 Functional Test Point PASS!!!
[1312000 ns] Test is running, debug_wb_pc = 0xbfc2ef90
----[1321725 ns] Number 8'd61 Functional Test Point PASS!!!
[1322000 ns] Test is running, debug_wb_pc = 0xbfc00ae8
[1332000 ns] Test is running, debug_wb_pc = 0xbfc4b900
[1342000 ns] Test is running, debug_wb_pc = 0xbfc4c7e8
----[1342985 ns] Number 8'd62 Functional Test Point PASS!!!
[1352000 ns] Test is running, debug_wb_pc = 0xbfc44da4
[1362000 ns] Test is running, debug_wb_pc = 0xbfc45cb4
----[1371175 ns] Number 8'd63 Functional Test Point PASS!!!
[1372000 ns] Test is running, debug_wb_pc = 0xbfc2ff28
[1382000 ns] Test is running, debug_wb_pc = 0xbfc30dd0
[1392000 ns] Test is running, debug_wb_pc = 0xbfc31cf8
----[1397715 ns] Number 8'd64 Functional Test Point PASS!!!
```

6. 总结

本次计算机系统实验是一次从理论到实践的深刻跨越。我们以“龙翼杯”竞赛为导向,面对仅支持四条指令的原始CPU模板,通过团队协作,最终成功构建了一个功能完备、支持 30 余条指令的五级流水线处理器。实验的核心成果不仅在于指令集的大幅扩展,更在于系统性地攻克了数据冒险、控制冒险及精细访存处理等关键技术难题,使CPU能够稳定、正确地执行所有基准测试程序。

在技术层面,我们的工作严格遵循了经典五级流水线架构,并对其进行了深度实现与优化。高荧同学构建了译码段(ID)的智慧核心,将指令支持扩展至完整的算术、逻辑、乘除、访存及跳转指令集,并设计了复杂的数据前递网络与Load-Use停顿检测机制,从源头保障了指令的正确解析与数据相关性处理。张诗琪同学夯实了执行段(EX)与访存段(MEM)的数据通路,实现了乘除法运算、HI/LO寄存器管理、字节对齐访存以及数据符号扩展等关键功能,为CPU提供了强大的计算与数据交互能力。贾培鑫同学则编织了控制段(CTRL)与顶层集成的神经网络,通过精确的全局停顿仲裁和清晰的模

块间互联,将各个独立的流水线阶段整合为一个协同、稳定的有机整体。

本次实验远超一次简单的代码编写任务,它是一次全面的工程能力锤炼。我们经历了从模块设计、代码实现、功能仿真到系统调试的完整开发流程。在调试过程中,面对波形图中错综复杂的信号与意料之外的错误,我们学会了如何冷静分析、逐级排查、定位根源,将课堂上学到的流水线冲突、时序等抽象概念转化为解决实际问题的具体方法。每一个测试点“PASS”的背后,都是对严谨工程思维和解决问题能力的一次提升。

尤为珍贵的是团队协作的经历。基于明确的分工与定期的代码审议,我们不仅高效完成了各自模块,更通过紧密的接口讨论与联合调试,深刻理解了系统级设计中模块间“握手”与数据流同步的重要性。这种从局部优化到全局集成的协作模式,让我们亲身体验了复杂硬件系统工程管理的真谛。

展望未来,本次实现的CPU仍留有广阔的探索空间。本次实验为我们揭开了处理器设计的神秘面纱,不仅巩固了计算机体系结构的核心知识,更点燃了我们深入探索计算机系统底层奥秘的热情。这段从零开始构建一台CPU的经历,将成为我们学习道路上坚实而难忘的里程碑。

附录

姓名	占比	主要负责模块	主要工作内容
高荧	40%	ID 段为主	1. 在 ID.v 中将指令译码逻辑从 4 条扩展至 30 余条，支持了算术、逻辑、访存、跳转及乘除法等指令。 2. 为解决数据冒险，在 ID.v 中设计并实现了完整的数据前递逻辑，将 EX/MEM/WB 段的结果旁路回 ID 段。 3. 针对 Load-Use 冲突，在 ID.v 中添加了 stallreq_for_load 停顿请求逻辑。 4. 相应地在 defines.vh 中扩展了流水线寄存器总线的位宽与控制字段。
张诗琪	32%	EX+MEM 段为主	1. 在 EX.v 中实现了 sb/sh/sw 指令的字节写使能生成与数据对齐逻辑。 2. 在 EX.v 中添加了 HI/LO 特殊寄存器，并实现了 mult/div 乘除法运算以及 div 指令的多周期停顿请求。 3. 在 MEM.v 中实现了 lw/lb/lbu/lh/ld 等加载指令的字节/半字选择与符号位/零扩展逻辑。
贾培鑫	28%	控制与顶层集成为主	1. 在 CTRL.v 中实现了全局停顿控制逻辑，能根据 ID 段和 EX 段的停顿请求，生成精确的 stall 总线信号以控制流水线。 2. 在顶层模块 mycpu_core.v 中调整了模块间的连线，将新增的停顿请求信号正确送入 CTRL 段。 3. 在 mycpu_core.v 中将后级流水线总线（ex_to_mem_bus, mem_to_wb_bus）回传至 ID 段，为数据前递提供了通路。